# PARTIE 3 — V3 : Prisma (ORM) avec Electron Forge + Vite

## Objectif

Vous partez de la **V2** (SQLite brut, fournie sur le dépôt). Vous allez y ajouter **Prisma ORM** pour remplacer le SQL brut par une couche d'abstraction typée.

Cette version utilise **Electron Forge + Vite** — un outil de build plus puissant que le simple `tsc` utilisé en V2. Vous allez comprendre pourquoi dans la section suivante.

**Ce qui change par rapport à V2 :**

- L'outil de build : `tsc` → Electron Forge + Vite
- La couche de persistance : SQL brut → Prisma ORM

**Ce qui reste identique :**

- Les composants Angular, les services, le preload
- Le schéma de données (`model Counter`)
- La logique IPC (mêmes 4 handlers)

---

## Théorie — Pourquoi Electron Forge + Vite ?

### Le problème du SQL brut (rappel V2)

En V2, chaque handler IPC ressemble à ceci :

```
ipcMain.handle('increment', (): number => {
    db.prepare('UPDATE Counter SET value = value + 1 WHERE id = 1').run();
    const row = db.prepare('SELECT value FROM Counter WHERE id = 1').get() as { value:
number };
    return row.value;
});
```

Les problèmes :

1. **Typage manuel** : `as { value: number }` — TypeScript ne vérifie pas que la colonne existe

2. **Évolution** : renommer `value` en `count` = chercher tous les `as { value: number }` dans le code
3. **Répétition** : chaque handler fait `prepare().run()` + `prepare().get()`
4. **Pas de traçabilité** : comment savoir qui a ajouté quelle colonne, et quand ?

## Qu'est-ce qu'un ORM ?

**ORM** = *Object-Relational Mapping*. Un ORM traduit les objets TypeScript en requêtes SQL, et vice-versa.

```
┌──────────────────────┐            ┌──────────────────────┐
│   Code TypeScript    │            │   Base de données    │
│                      │    ORM     │                      │
│  counter.value = 42  │ ◄────────► │  UPDATE Counter      │
│  (objet TypeScript)  │            │  SET value = 42      │
│                      │            │  WHERE id = 1        │
└──────────────────────┘            └──────────────────────┘
```

**Sans ORM (SQL brut) :**

```
const row = db.prepare('SELECT value FROM Counter WHERE id = 1').get() as { value:
number };
// type : any — pas de vérification à la compilation
```

**Avec Prisma :**

```
const counter = await prisma.counter.findUnique({ where: { id: 1 } });
// counter.value : type 'number' — TypeScript le sait grâce au schéma
```

## Les 3 composants de Prisma

```
┌──────────────────────────────────────────────────────────────────┐
│                           PRISMA                                 │
│                                                                  │
│  ┌──────────────────┐   ┌──────────────────┐   ┌──────────────┐ │
│  │ Prisma Schema    │   │ Prisma Migrate   │   │ Prisma       │ │
│  │ schema.prisma    │   │ (CLI)            │   │ Client       │ │
│  │                  │   │                  │   │              │ │
│  │ Décrit les       │   │ Génère et        │   │ L'API        │ │
│  │ modèles de       │──►│ applique les     │   │ TypeScript   │ │
│  │ données          │   │ migrations SQL   │──►│ pour         │ │
│  │ (source de       │   │                  │   │ interroger   │ │
│  │ vérité)          │   │                  │   │ la DB        │ │
│  └──────────────────┘   └──────────────────┘   └──────────────┘ │
└──────────────────────────────────────────────────────────────────┘
```

### Pourquoi Vite est nécessaire avec Prisma v7

Prisma v7 a changé son générateur. L'ancien (`prisma-client-js`) produisait du code CJS. Le nouveau (`prisma-client-ts`) produit du **TypeScript ESM pur**, avec des `import.meta.url` — une syntaxe qui n'existe pas en CJS.

JavaScript a deux systèmes de modules :

```
┌──────────────────────────────────────────────────────────────┐
│                      SYSTÈMES DE MODULES                     │
│                                                              │
│  ┌──────────────────────────┐    ┌──────────────────────────┐  │
│  │  CJS — CommonJS          │    │  ESM — ES Modules        │  │
│  │                          │    │                          │  │
│  │  require('./foo')        │    │  import foo from './foo' │  │
│  │  module.exports = {}     │    │  export default {}       │  │
│  │                          │    │                          │  │
│  │  Node.js historique      │    │  Standard moderne        │  │
│  │  Electron preload        │    │  import.meta.url disponible │  │
│  └──────────────────────────┘    └──────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

Le **preload Electron doit être CJS** — contrainte imposée par Electron. `tsc` compile chaque fichier séparément et ne peut pas convertir `import.meta.url` en CJS. D'où le conflit.

**Vite est un bundler.** Au lieu de compiler fichier par fichier, il analyse toutes les dépendances et produit **un seul fichier CJS** :

```
AVEC tsc (V2) — fichier par fichier
──────────────────────────────────────────────────────
  src/main.ts          →  dist/main.js        ✔
  import.meta.url      →  reste tel quel      ✗ ERREUR

AVEC Vite (V3) — tout bundlé
──────────────────────────────────────────────────────
  src/main/main.ts ──┐
  src/prisma/...   ──►│  Vite bundle  →  .vite/build/main.js  ✔
  node_modules/...──┘  (tout en CJS)   import.meta → __dirname
```

Vite traduit automatiquement `import.meta.url` en `__dirname` pendant le bundle. Le conflit ESM/CJS disparaît.

### Prisma dans Electron

Prisma ne tourne que dans le **Main Process** (Node.js). Le Renderer Angular ne peut pas importer `@prisma/client`.

La communication se fait toujours via **IPC** :

```
Angular          Preload           Main Process        SQLite
   |                |                    |                 |
   |—increment()——►|                    |                 |
   |                |—invoke('incr')——►|                 |
   |                |                    |—prisma.update()►|
   |                |                    |◄——43————————|
   |                |◄——43————————————|                 |
   |◄——43————————|                    |                 |
count.set(43)       |                    |                 |
```

---

## Étape 01 — Créer le projet Electron Forge

### Scaffolding

Dans le terminal, dans votre dossier de travail :

```
npm create electron-app@latest my-counter-v3 -- --template=vite-typescript
cd my-counter-v3
```

Le flag `--template=vite-typescript` génère un projet Electron Forge préconfiguré avec Vite et TypeScript pour le main et le preload.

> **Electron Forge** : au début du cours, cet outil avait été présenté comme "trop de fichiers de configuration". Maintenant vous comprenez pourquoi cette complexité existe — `forge.config.ts` décrit comment compiler chaque partie de l'application, et `VitePlugin` garantit que le bundle final est CJS.

### Vérifier que le projet de base démarre

```
npm start
```

Une fenêtre Electron s'ouvre avec "Hello World". Fermez-la et continuez.

### Réorganiser la structure des dossiers

On va créer un dossier main et preload : `main.ts` et `preload.ts` qu'on reprendra directement du précendant projet.

On les déplace dans des sous-dossiers :

```
mkdir -p src/main src/preload
```

Mettre à jour `forge.config.ts` pour pointer vers les nouveaux chemins :

```
// forge.config.ts — modifier les entrées du VitePlugin
new VitePlugin({
  build: [
    {
      entry: 'src/main/main.ts',       // ← chemin mis à jour
      config: 'vite.main.config.mts',
      target: 'main',
    },
    {
      entry: 'src/preload/preload.ts', // ← chemin mis à jour
      config: 'vite.preload.config.mts',
      target: 'preload',
    },
  ],
  renderer: [],
}),
```

### Structure cible

```
my-counter-v3/
├── forge.config.ts
├── vite.main.config.mts
├── vite.preload.config.mts
├── prisma/
│   └── schema.prisma
├── prisma.config.ts
├── .env
├── src/
│   ├── main/
│   │   └── main.ts
│   ├── preload/
│   │   └── preload.ts
│   └── prisma/
│       └── generated/     ← généré automatiquement par prisma generate
```

```
└── renderer/
    └── app/                ← application Angular (copiée de V2)
```

---

## Étape 02 — Copier le renderer Angular

Le renderer Angular de la V2 est réutilisé.

### Copier le dossier

```
cp -r ../v2-sqlite/renderer ./renderer
```

### Builder le renderer

```
cd renderer/app
npm i
ng build
```

Si le build échoue avec :

```
[ERROR] TS2307: Cannot find module '../../../types/electron' or its corresponding type
declarations.
```

Le chemin d'import vers `electron.ts` dans votre service est incorrect. Ouvrez `electron.service.ts` et corrigez le chemin relatif — par exemple `../../types/electron` à la place de `../../../types/electron`.

Le build réussi produit `renderer/app/dist/app/browser/index.html`.

---

## Étape 03 — Installer Prisma

Retourner à la racine du projet Electron :

```
cd ../..
```

```
npm install typescript@latest --save-dev
```

```
npm install @prisma/client @prisma/adapter-better-sqlite3 better-sqlite3 dotenv
npm install --save-dev prisma @types/better-sqlite3
```

| Package | Type | Rôle |
|---|---|---|
| `@prisma/client` | dependency | Client Prisma (types + runtime) |
| `@prisma/adapter-better-sqlite3` | dependency | Driver qui connecte Prisma à SQLite |
| `better-sqlite3` | dependency | Module SQLite synchrone pour Node.js |
| `dotenv` | dependency | Lecture du fichier `.env` par `prisma.config.ts` |
| `prisma` | devDependency | CLI Prisma (`migrate`, `generate`, `studio`) |
| `@types/better-sqlite3` | devDependency | Types TypeScript pour `better-sqlite3` |

Ajouter les scripts Prisma dans `package.json` :

```
{
  "scripts": {
    "start":           "electron-forge start",
    "build:angular":   "cd renderer/app && ng build",
    "package":         "electron-forge package",
    "make":            "electron-forge make",
    "prisma:generate": "npx prisma generate",
    "prisma:migrate":  "npx prisma migrate dev --name init",
    "prisma:studio":   "npx prisma studio"
  }
}
```

---

## Étape 04 — Initialiser Prisma

### `prisma init`

Depuis la racine du projet, lancer :

```
npx prisma init --datasource-provider sqlite
```

Cette commande crée automatiquement :

```
my-counter-v3/
├── prisma/
│   └── schema.prisma   ← schéma de base (SQLite configuré, pas d'url)
└── prisma.config.ts    ← configuration CLI pour Prisma v7
```

Elle tente aussi d'écrire `DATABASE_URL` dans `.env`. Si le fichier existe déjà, elle affiche un avertissement et ne l'écrase pas.

### Le fichier `.env`

Vérifier que `.env` contient bien :

```
DATABASE_URL="file:./counter.db"
```

Si le fichier n'existe pas encore, le créer et y coller cette ligne.

Ce fichier est lu par la CLI Prisma (`migrate`, `studio`) pour localiser la base. Il ne doit **jamais** être commité dans git (déjà dans `.gitignore`).

### Le fichier `prisma.config.ts` généré

`prisma init` crée déjà le bon fichier pour Prisma :

```typescript
// prisma.config.ts — généré par prisma init
import 'dotenv/config';
import { defineConfig } from 'prisma/config';

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
  },
  datasource: {
    url: process.env['DATABASE_URL'], // ← lit depuis .env
  },
});
```

**Aucune modification nécessaire.**

| Fichier | Lu par | Rôle |
|---|---|---|
| `.env` | `prisma.config.ts` | URL pour la CLI (`migrate`, `studio`) |
| `prisma.config.ts` | CLI Prisma | Configuration complète de la CLI |

| Fichier | Lu par | Rôle |
|---|---|---|
| Driver adapter dans `main.ts` | Electron | URL pour l'exécution de l'app |

## Étape 05 — Écrire le schéma Prisma

Le dossier `prisma/` et le fichier `schema.prisma` ont été créés par `prisma init`. Il faut maintenant **remplacer leur contenu** par le schéma définitif.

Ouvrir `prisma/schema.prisma` et remplacer tout le contenu par :

```
// prisma/schema.prisma

generator client {
  provider = "prisma-client-ts"
  output   = "../src/prisma/generated"
}

datasource db {
  provider = "sqlite"
}

model Counter {
  id    Int @id @default(1)
  value Int @default(0)
}
```

### Anatomie du modèle

| Champ | Type SQL | Type TypeScript | Attributs | Signification |
|---|---|---|---|---|
| `id` | `INTEGER` | `number` | `@id @default(1)` | Clé primaire, valeur fixe 1 |
| `value` | `INTEGER` | `number` | `@default(0)` | Valeur du compteur, 0 par défaut |

**Pourquoi `id = 1` fixe ?** Notre application n'a besoin que d'un seul compteur. En fixant l'id à 1, on garantit qu'il n'y en a jamais plus d'un. Pour plusieurs compteurs indépendants, on utiliserait `@default(autoincrement())`.

### Les types Prisma

| Type Prisma | Type SQL | Type TypeScript |
|---|---|---|
| `Int` | INTEGER | `number` |
| `String` | TEXT | `string` |
| `Boolean` | BOOLEAN | `boolean` |
| `Float` | REAL | `number` |
| `DateTime` | DATETIME | `Date` |

### Les attributs de champ

| Attribut | Signification | Exemple |
|---|---|---|
| `@id` | Clé primaire | `id Int @id` |
| `@default(val)` | Valeur par défaut | `@default(0)` |
| `@default(autoincrement())` | Auto-incrément | `id Int @id @default(autoincrement())` |
| `@default(now())` | Date actuelle à la création | `createdAt DateTime @default(now())` |
| `@unique` | Valeur unique dans la colonne | `email String @unique` |
| `@updatedAt` | Mis à jour automatiquement | `updatedAt DateTime @updatedAt` |
| `?` après le type | Champ optionnel (nullable) | `description String?` |

### Ce que Prisma va générer

Prisma traduira ce modèle en SQL lors de la migration :

```sql
CREATE TABLE "Counter" (
    "id"    INTEGER NOT NULL DEFAULT 1,
    "value" INTEGER NOT NULL DEFAULT 0,
    CONSTRAINT "Counter_pkey" PRIMARY KEY ("id")
);
```

Vous n'avez **jamais** à écrire ce SQL vous-même.

Vérifier que le schéma est valide avant de migrer :

```
npx prisma validate
```

### Le générateur `prisma-client-ts`

En Prisma v7, le générateur produit des fichiers TypeScript directement dans votre projet, dans `src/prisma/generated/` :

```
src/prisma/generated/
├── client.ts           ← PrismaClient — à importer dans main.ts
├── models.ts           ← types générés (Counter, etc.)
├── models/
│   └── Counter.ts      ← type Counter
└── internal/
    └── ...
```

---

## Étape 06 — La première migration

### Qu'est-ce qu'une migration ?

Une **migration** est un fichier SQL versionné qui décrit un changement de schéma. Chaque modification de `schema.prisma` crée une nouvelle migration enregistrée dans `prisma/migrations/`.

```
prisma/
├── schema.prisma
└── migrations/
    ├── 20241201_120000_init/       ← migration n°1
    │   └── migration.sql
    └── 20241215_140000_add_name/   ← migration n°2 (future)
        └── migration.sql
```

L'historique permet de :

- **Tracer** l'évolution du schéma
- **Synchroniser** la base entre développeurs
- **Revenir en arrière** si nécessaire

### Créer et appliquer la migration

```
npm run prisma:migrate
npm run prisma:generate
```

`prisma migrate dev` exécute :

1. Lit `prisma/schema.prisma` et `prisma.config.ts`
2. Compare avec l'état actuel de la base
3. Génère le SQL dans `prisma/migrations/.../migration.sql`
4. Applique le SQL sur `counter.db`

> **Important :** `migrate` ne génère le client TypeScript que lors d'une vraie migration (schéma modifié). Si la base est déjà en sync, il ne produit rien dans `src/prisma/generated/`. Toujours lancer `prisma:generate` après `prisma:migrate` pour s'assurer que le client existe.

`prisma generate` produit le Prisma Client TypeScript dans `src/prisma/generated/`.

**Résultat attendu après migrate :**

```
Your database is now in sync with your schema.
```

> Si vous relancez sans avoir modifié `schema.prisma`, Prisma indique "Already in sync, no schema change or pending migration was found." — c'est normal.

### Visualiser la base avec Prisma Studio

```
npm run prisma:studio
```

Ouvre `http://localhost:xxxxx` — interface graphique pour voir et modifier les données de `counter.db`.

### ✔ Checkpoint — Vérifier la migration avec SQLite

Avant de continuer, vérifiez que la table a bien été créée avec l'extension **SQLite** (alexcvzz) dans VS Code.

**Inspecter `counter.db` :**

1. Vous devez voir la table `Counter` avec les colonnes `id` et `value`
2. La table est **vide** pour l'instant — c'est normal, la ligne sera créée au premier démarrage de l'application (via `upsert`)

```
counter.db — SQLite Viewer
┌──────────────┐
│  Counter     │
├────┬─────────┤
│ id │  value  │
├────┼─────────┤
│  (vide)      │
└──────────────┘
```

Si la table `Counter` apparaît → la migration a réussi. Passez à l'Étape 07. Si `counter.db` n'existe pas ou est vide → relancez `npm run prisma:migrate`.

---

## Étape 07 — Le Prisma Client

Après `npm run prisma:generate`, le Prisma Client est disponible dans `src/prisma/generated/` :

```
src/prisma/generated/
├── client.ts           ← PrismaClient — à importer dans main.ts
├── models.ts           ← types générés (Counter, etc.)
├── models/
│   └── Counter.ts      ← type Counter
└── internal/
    └── ...
```

Il expose des méthodes pour chaque modèle défini dans `schema.prisma`, détaillées ci-dessous.

### Lire — `findUnique`

```
const counter = await prisma.counter.findUnique({
  where: { id: 1 }
});
// counter : Counter | null
// counter.value : number — TypeScript le sait
```

### Lire plusieurs — `findMany`

```
const tous = await prisma.counter.findMany();
// avec filtres, tris, limites :
const todos = await prisma.todo.findMany({
  where:   { done: false },
  orderBy: { createdAt: 'desc' },
  take:    10,
});
```

### Mettre à jour — `update`

```
// Valeur directe
const modifie = await prisma.counter.update({
  where: { id: 1 },
  data:  { value: 42 }
});

// Opérations atomiques sur les nombres (pas de race condition) :
await prisma.counter.update({
  where: { id: 1 },
  data:  { value: { increment: 1 } }   // value = value + 1
});

await prisma.counter.update({
  where: { id: 1 },
  data:  { value: { decrement: 1 } }
});

await prisma.counter.update({
  where: { id: 1 },
  data:  { value: { multiply: 2 } }
});
```

### Créer ou mettre à jour — `upsert`

```
await prisma.counter.upsert({
  where:  { id: 1 },          // cherche cet enregistrement
  update: {},                  // si trouvé : ne rien modifier
  create: { id: 1, value: 0 } // si non trouvé : créer avec ces valeurs
});
```

`upsert` est idéal pour l'**initialisation** : on s'assure que l'enregistrement existe sans l'écraser s'il existe déjà.

### Connexion et déconnexion

```
const prisma = new PrismaClient({ adapter }); // une seule instance pour toute l'app

// À la fermeture de l'application Electron :
app.on('before-quit', async () => {
  await prisma.$disconnect();
});
```

---

# Étape 08 — better-sqlite3

### `better-sqlite3` est un module natif

`better-sqlite3` contient du code C++ compilé en un fichier `.node`. Ce fichier est spécifique à la version de Node.js et au système d'exploitation.

Electron embarque **sa propre version de Node.js**, différente de celle installée sur votre machine. Il faut donc que `better-sqlite3` soit compilé pour la version de Node.js d'Electron, pas pour celle du système.

Electron Forge s'en charge automatiquement au démarrage — vous n'avez rien à faire.

### ⚠️ Piège : `better-sqlite3` et Vite

Par défaut, Vite tente d'inclure **toutes** les dépendances dans le bundle — y compris `better-sqlite3`. Or ce module contient un binaire natif compilé (`better_sqlite3.node`) qui **ne peut pas être bundlifié**. Si vous laissez `vite.main.config.ts` vide, vous obtiendrez cette erreur au lancement :

```
Error: Could not dynamically require "...better_sqlite3.node".
Please configure the dynamicRequireTargets or/and ignoreDynamicRequires option
of @rollup/plugin-commonjs appropriately for this require call to work.
```

**Correction obligatoire** — ouvrir `vite.main.config.ts` et y ajouter :

```
import { defineConfig } from 'vite';

export default defineConfig({
  build: {
    rollupOptions: {
      external: [
        'better-sqlite3',
        /\.node$/,
      ],
    },
  },
});
```

Cela indique à Vite de laisser `better-sqlite3` et tous les fichiers `.node` **en dehors** du bundle. Electron les chargera directement depuis `node_modules` au runtime.

> Sans cette configuration, l'application plante au démarrage avant même d'afficher la fenêtre.

### Résultat du build Vite

Après `npm start`, Vite produit dans `.vite/build/` :

```
.vite/build/
├── main.js      ← main process bundlé (CJS, tout inclus)
└── preload.js   ← preload bundlé (CJS)
```

Ces deux fichiers sont **au même niveau** dans `.vite/build/`. C'est important pour les chemins dans `main.ts`.

---

## Étape 09 — Intégrer Prisma dans `src/main/main.ts`

### 1 — Les imports et le driver adapter

```
import { app, BrowserWindow, ipcMain } from 'electron';
import path from 'node:path';
import started from 'electron-squirrel-startup';
import { PrismaBetterSqlite3 } from '@prisma/adapter-better-sqlite3';
import { PrismaClient } from '../prisma/generated/client.js';
//                          ↑ chemin vers le client généré dans src/prisma/generated/
//                            l'extension .js est requise (convention ESM)

// Quitter immédiatement lors de l'installation/désinstallation Windows
if (started) app.quit();
```

### 2 — Le chemin de la base et le driver adapter

```
// Vite place le bundle dans .vite/build/main.js
// __dirname = .vite/build/
// ../.. remonte à la racine du projet (où se trouve counter.db)
const dbPath = path.join(__dirname, '..', '..', 'counter.db');

const adapter = new PrismaBetterSqlite3({ url: 'file:' + dbPath });
const prisma = new PrismaClient({ adapter });
//                                ↑ on passe le driver adapter au client
//                                  Prisma n'a plus besoin de process.env
```

> `__dirname` vaut `.vite/build/` après le bundle Vite. Le `../..` remonte à la racine du projet où se trouve `counter.db`.

### 3 — La fonction d'initialisation

`upsert` garantit qu'une ligne existe sans jamais l'écraser si elle existe déjà :

```
async function initCounter(): Promise<void> {
    await prisma.counter.upsert({
        where:  { id: 1 },
        update: {},                  // si trouvé : ne rien changer
        create: { id: 1, value: 0 }, // si absent : créer avec value = 0
    });
}
```

### 4 — La fenêtre et le cycle de vie

```
function createWindow(): void {
    const win = new BrowserWindow({
        width: 500,
        height: 450,
        webPreferences: {
            preload: path.join(__dirname, 'preload.js'),
            // ↑ même dossier que main.js — .vite/build/preload.js
            contextIsolation: true,
        },
    });

    // Charge le renderer Angular (buildé avec ng build)
    win.loadFile(
        path.join(__dirname, '..', '..', 'renderer/app/dist/app/browser/index.html')
    );
}

app.whenReady().then(async () => { // async requis pour pouvoir utiliser await
    await initCounter();           // créer le compteur si absent
    createWindow();
});

app.on('before-quit', async () => {
    await prisma.$disconnect();    // fermer la connexion proprement
});
```

## 5 — Les IPC handlers

Chaque handler devient `async` et retourne le résultat de la requête Prisma. TypeScript sait déjà que `counter.value` est un `number` — c'est Prisma qui lui a transmis cette information en lisant le schéma. Pas besoin de cast :

```
ipcMain.handle('get-counter', async (): Promise<number> => {
    const counter = await prisma.counter.findUnique({ where: { id: 1 } });
    return counter?.value ?? 0;
});

ipcMain.handle('increment', async (): Promise<number> => {
    const counter = await prisma.counter.update({
        where: { id: 1 },
        data:  { value: { increment: 1 } }, // opération atomique SQLite
    });
    return counter.value; // TypeScript sait que c'est un number
});

ipcMain.handle('decrement', async (): Promise<number> => {
    const counter = await prisma.counter.update({
        where: { id: 1 },
        data:  { value: { decrement: 1 } },
    });
    return counter.value;
});

ipcMain.handle('reset', async (): Promise<number> => {
    const counter = await prisma.counter.update({
        where: { id: 1 },
        data:  { value: 0 },
    });
    return counter.value;
});
```

## Le fichier `main.ts` complet

```
// src/main/main.ts
import { app, BrowserWindow, ipcMain } from 'electron';
import path from 'node:path';
import started from 'electron-squirrel-startup';
import { PrismaBetterSqlite3 } from '@prisma/adapter-better-sqlite3';
import { PrismaClient } from '../prisma/generated/client.js';

if (started) app.quit();

const dbPath = path.join(__dirname, '..', '..', 'counter.db');
const adapter = new PrismaBetterSqlite3({ url: 'file:' + dbPath });
```

```
const prisma = new PrismaClient({ adapter });

async function initCounter(): Promise<void> {
    await prisma.counter.upsert({
        where:  { id: 1 },
        update: {},
        create: { id: 1, value: 0 },
    });
}

function createWindow(): void {
    const win = new BrowserWindow({
        width: 500,
        height: 450,
        webPreferences: {
            preload: path.join(__dirname, 'preload.js'),
            contextIsolation: true,
        },
    });
    win.loadFile(
        path.join(__dirname, '..', '..', 'renderer/app/dist/app/browser/index.html')
    );
}

app.whenReady().then(async () => {
    await initCounter();
    createWindow();
});

app.on('before-quit', async () => {
    await prisma.$disconnect();
});

ipcMain.handle('get-counter', async (): Promise<number> => {
    const counter = await prisma.counter.findUnique({ where: { id: 1 } });
    return counter?.value ?? 0;
});

ipcMain.handle('increment', async (): Promise<number> => {
    const counter = await prisma.counter.update({
        where: { id: 1 },
        data:  { value: { increment: 1 } },
    });
    return counter.value;
});

ipcMain.handle('decrement', async (): Promise<number> => {
    const counter = await prisma.counter.update({
        where: { id: 1 },
        data:  { value: { decrement: 1 } },
    });
    return counter.value;
});
```

```
ipcMain.handle('reset', async (): Promise<number> => {
    const counter = await prisma.counter.update({
        where: { id: 1 },
        data:  { value: 0 },
    });
    return counter.value;
});
```

---

## Étape 10 — Le preload

Le contenu de `src/preload/preload.ts` est **identique à la V2**. Vite le compile automatiquement en CJS — vous n'avez rien à configurer.

> ⚠️ **Attention — structure différente entre V2 et V3 :** En V2, le preload se trouve à `src/preload.ts` (directement dans `src/` ). En V3, il se trouve à `src/preload/preload.ts` (dans un sous-dossier). La commande `cp -r` de l'Étape 01 ne copie pas ce fichier automatiquement. **Copiez le contenu manuellement** ou recréez le fichier avec le code ci-dessous.

Ouvrir `src/preload/preload.ts` et y écrire :

```
// src/preload/preload.ts
import { contextBridge, ipcRenderer } from 'electron';

contextBridge.exposeInMainWorld('api', {
    getCounter: (): Promise<number> => ipcRenderer.invoke('get-counter'),
    increment:  (): Promise<number> => ipcRenderer.invoke('increment'),
    decrement:  (): Promise<number> => ipcRenderer.invoke('decrement'),
    reset:      (): Promise<number> => ipcRenderer.invoke('reset'),
});
```

**Rappel :** `contextBridge.exposeInMainWorld` crée un pont sécurisé — Angular ne peut appeler que les méthodes explicitement listées ici, pas d'accès direct à Node.js.

> **Si le preload reste vide**, `window.api` n'existe jamais dans Angular. Tous les clics sur les boutons échouent silencieusement — l'interface ne répond pas.

## Étape 11 — Les services Angular (inchangés)

**Action :** ne modifiez pas `electron.service.ts` ni `counter.service.ts`. Vérifiez simplement que vos fichiers correspondent à ce qui suit.

Les services sont **identiques à la V2**.

```
// electron.service.ts — inchangé
@Injectable({ providedIn: 'root' })
export class ElectronService {
  isElectron(): boolean { return !!(window && window.api); }
  getApi(): CounterAPI {
    if (!this.isElectron()) throw new Error('API Electron non disponible');
    return window.api;
  }
}

// counter.service.ts — inchangé
@Injectable({ providedIn: 'root' })
export class CounterService {
  constructor(private electronService: ElectronService) {}
  getCounter(): Promise<number> { return this.electronService.getApi().getCounter(); }
  increment():  Promise<number> { return this.electronService.getApi().increment(); }
  decrement():  Promise<number> { return this.electronService.getApi().decrement(); }
  reset():      Promise<number> { return this.electronService.getApi().reset(); }
}
```

Les services ne savent pas si la persistance est JSON, SQLite brut ou Prisma. Seul `main.ts` côté Electron change — c'est le principe de séparation des responsabilités.

## Étape 12 — Builder et lancer

```
RAPPEL :
# 1. Builder le renderer Angular (à faire une fois, ou après chaque modification
Angular)
npm run build:angular

# 2. Créer/appliquer la migration SQLite (une seule fois, ou après chaque modif du
schéma)
npm run prisma:migrate
# ↑ génère aussi le client TypeScript dans src/prisma/generated/ automatiquement

# Si vous clonez le projet sans migrer (ex : DB déjà fournie par le prof)
npm run prisma:generate

Maintenant Lancer l'application
npm start
```

`npm start` enchaîne automatiquement :

1. Vite bundle `src/main/main.ts` → `.vite/build/main.js`
2. Vite bundle `src/preload/preload.ts` → `.vite/build/preload.js`
3. `npx electron .` → lance Electron

**Vérifier la persistance :**

1. Cliquer plusieurs fois sur "+" (ex : compteur à 5)
2. Fermer l'application
3. Relancer → le compteur affiche toujours **5**
4. Un fichier `counter.db` est présent à la racine du projet

---

### ✔ Checkpoint — Vérifier la persistance avec SQLite

Après avoir cliqué plusieurs fois sur "+" et fermé l'application :

1. Dans VS Code, clic sur `counter.db`
2. SQLite Viewer affiche la table `Counter`
3. Vous devez voir la ligne `{ id: 1, value: 5 }` (ou la valeur que vous avez atteinte)

```
counter.db — SQLite
├────┼──────────┤
│  1 │    5     │  ← persisté en base via Prisma
```

Si la valeur correspond à ce que vous avez affiché dans l'application → Prisma écrit bien dans SQLite. La persistance fonctionne.

---

| Script | Commande | Utilité |
|---|---|---|
| `npm start` | `electron-forge start` | Bundle Vite + lance Electron |
| `npm run build:angular` | `cd renderer/app && ng build` | Compile le renderer Angular |
| `npm run package` | `electron-forge package` | Crée l'exécutable distributable |
| `npm run prisma:migrate` | `prisma migrate dev` | Créer/appliquer les migrations |
| `npm run prisma:generate` | `prisma generate` | Régénérer le client TypeScript |
| `npm run prisma:studio` | `prisma studio` | Interface visuelle de la base |

---

## Étape 13 — Les composants enfants (inchangés)

**Action :** ne modifiez pas `counter-display` ni `counter-buttons`. Ils sont identiques à la V2 — ils n'ont aucune idée de la couche de persistance.

```
// counter-display.ts — inchangé
export class CounterDisplay {
  @Input() value!: number;
}

// counter-buttons.ts — inchangé
export class CounterButtons {
  @Output() incremented = new EventEmitter<void>();
  @Output() decremented = new EventEmitter<void>();
  @Output() resetted    = new EventEmitter<void>();

  onIncrement(): void { this.incremented.emit(); }
  onDecrement(): void { this.decremented.emit(); }
  onReset():     void { this.resetted.emit(); }
}
```

| Décorateur | Direction | Déclaration | Utilisation dans le parent |
|---|---|---|---|
| `@Input()` | Parent → Enfant | `@Input() value!: number` | `[value]="count()"` |

| Décorateur | Direction | Déclaration | Utilisation dans le parent |
|---|---|---|---|
| `@Output()` | Enfant → Parent | `@Output() incremented = new EventEmitter<void>()` | `(incremented)="increment()"` |

---

### Flux complet d'un clic sur "+" — Question d'examen !

```
Clic sur <button> dans CounterButtons
    │
    ▼ (click)="onIncrement()"
CounterButtons.onIncrement()
    │
    ▼ this.incremented.emit()
@Output incremented émet vers App
    │
    ▼ (incremented)="increment()"
App.increment()
    │
    ▼ await this.counterService.increment()
CounterService → ElectronService → window.api
    │
    ▼ ipcRenderer.invoke('increment')
main.ts : prisma.counter.update({ value: { increment: 1 } })
SQLite : UPDATE Counter SET value = value + 1 WHERE id = 1
    │
    ▼ retourne 43
count.set(43)
    │
    ▼ Angular propage [value]="count()" vers CounterDisplay
<div class="value">43</div>
```

## Évolution du schéma — exemple pratique

Supposons qu'on veuille ajouter un nom au compteur :

**1. Modifier `schema.prisma` :**

```
model Counter {
  id    Int    @id @default(1)
  value Int    @default(0)
  name  String @default("Mon compteur")  // ← nouveau champ
}
```

**2. Créer une nouvelle migration :**

```
npx prisma migrate dev --name add_name_to_counter
npm run prisma:generate  (pour regénérer le model dans prisma/generated/models)
```

Prisma génère automatiquement :

```
ALTER TABLE "Counter" ADD COLUMN "name" TEXT NOT NULL DEFAULT 'Mon compteur';
```

**3. Le type TypeScript est automatiquement mis à jour dans `src/prisma/generated/` :**

```
const counter = await prisma.counter.findUnique({ where: { id: 1 } });
counter.name  // TypeScript sait maintenant que ce champ existe
```

---

## Pour aller plus loin — Le pattern Repository

À ce stade, `main.ts` mélange deux responsabilités :

- La **fenêtre Electron** (créer la `BrowserWindow`, gérer les événements `app`)
- Les **accès base de données** (toutes les requêtes Prisma)

C'est acceptable pour un compteur. Dès que le projet grandit (plusieurs tables, plusieurs handlers), `main.ts` devient difficile à lire et à maintenir.

La solution standard est d'isoler les accès DB dans un fichier dédié : le **repository**.

### Structure

```
src/
├── main/
│   ├── main.ts                        ← fenêtre + IPC handlers (allégé)
│   └── repository/
│       └── counter.repository.ts      ← toutes les requêtes Prisma
└── preload/
    └── preload.ts
```

### `src/main/repository/counter.repository.ts`

```
import { PrismaBetterSqlite3 } from '@prisma/adapter-better-sqlite3';
import { PrismaClient } from '../../prisma/generated/client.js';
import { app } from 'electron';
import path from 'node:path';

// app.getPath('userData') = répertoire accessible en lecture/écriture
// En dev    : ~/Library/Application Support/my-counter-v3/
// En prod   : même chemin — hors de l'archive .asar
const dbPath = path.join(app.getPath('userData'), 'counter.db');
const adapter = new PrismaBetterSqlite3({ url: 'file:' + dbPath });
const prisma = new PrismaClient({ adapter });

// Crée la table si elle n'existe pas encore (équivalent de prisma migrate deploy)
async function runMigrations(): Promise<void> {
    await prisma.$executeRawUnsafe(`
        CREATE TABLE IF NOT EXISTS "Counter" (
            "id"    INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT DEFAULT 1,
            "value" INTEGER NOT NULL DEFAULT 0,
            "name"  TEXT    NOT NULL DEFAULT 'test'
        )
    `);
}

export async function initCounter(): Promise<void> {
    await runMigrations();
    await prisma.counter.upsert({
        where:  { id: 1 },
        update: {},
        create: { id: 1, value: 0 },
    });
}

export async function getCounter(): Promise<number> {
    const counter = await prisma.counter.findUnique({ where: { id: 1 } });
    return counter?.value ?? 0;
}

export async function increment(): Promise<number> {
```

```
    const counter = await prisma.counter.update({
        where: { id: 1 },
        data:  { value: { increment: 1 } },
    });
    return counter.value;
}

export async function decrement(): Promise<number> {
    const counter = await prisma.counter.update({
        where: { id: 1 },
        data:  { value: { decrement: 1 } },
    });
    return counter.value;
}

export async function reset(): Promise<number> {
    await prisma.counter.update({ where: { id: 1 }, data: { value: 0 } });
    return 0;
}

export async function closeDb(): Promise<void> {
    await prisma.$disconnect();
}
```

### `src/main/main.ts` — allégé

```
import { app, BrowserWindow, ipcMain } from 'electron';
import path from 'node:path';
import started from 'electron-squirrel-startup';
import * as db from './repository/counter.repository';

if (started) app.quit();

function createWindow(): void {
    const win = new BrowserWindow({
        width: 500,
        height: 450,
        webPreferences: {
            preload: path.join(__dirname, 'preload.js'),
            contextIsolation: true,
        },
    });
    // En dev : MAIN_WINDOW_VITE_DEV_SERVER_URL est défini par le plugin Vite
    // En prod : on charge le fichier HTML généré par Vite dans .vite/renderer/
    if (MAIN_WINDOW_VITE_DEV_SERVER_URL) {
        win.loadURL(MAIN_WINDOW_VITE_DEV_SERVER_URL);
    } else {
        win.loadFile(
            path.join(__dirname, '..', `renderer/${MAIN_WINDOW_VITE_NAME}/index.html`)
        );
    }
```

```
}

app.whenReady().then(async () => {
    await db.initCounter();
    createWindow();
});

app.on('before-quit', async () => { await db.closeDb(); });

ipcMain.handle('get-counter', () => db.getCounter());
ipcMain.handle('increment',   () => db.increment());
ipcMain.handle('decrement',   () => db.decrement());
ipcMain.handle('reset',       () => db.reset());
```

Les variables `MAIN_WINDOW_VITE_DEV_SERVER_URL` et `MAIN_WINDOW_VITE_NAME` sont injectées automatiquement par `@electron-forge/plugin-vite` au moment du build. Pour que TypeScript les accepte sans erreur, créer le fichier de déclaration `src/main/vite-env.d.ts` :

```
// src/main/vite-env.d.ts
declare const MAIN_WINDOW_VITE_DEV_SERVER_URL: string | undefined;
declare const MAIN_WINDOW_VITE_NAME: string;
```

---

## Problèmes courants

### `Cannot find module '../prisma/generated/client.js'`

Le client n'a pas été généré :

```
npm run prisma:generate
```

Vérifier que `src/prisma/generated/client.ts` existe.

### `Table "Counter" does not exist`

La migration n'a pas été appliquée :

```
npm run prisma:migrate
```

### La fenêtre s'ouvre mais Angular ne charge pas

Le build Angular n'a pas été lancé :

```
npm run build:angular
```

Vérifier que `renderer/app/dist/app/browser/index.html` existe.

### La valeur ne persiste pas entre les lancements

Vérifier le chemin de la base dans `main.ts` :

```
const dbPath = path.join(__dirname, '..', '..', 'counter.db');
// __dirname = .vite/build/ → ../.. = racine du projet
```

Vérifier qu'un fichier `counter.db` existe à la racine du projet.

### `signal.set()` ne met pas à jour l'interface

Vérifier que `provideZonelessChangeDetection()` est dans `app.config.ts` :

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZonelessChangeDetection(), // ← obligatoire
  ]
};
```

### `Property 'api' does not exist on type 'Window'`

Vérifier que `electron.ts` (et non `electron.d.ts`) est bien dans `renderer/app/src/app/types/` et que l'import dans `electron.service.ts` pointe vers `'./types/electron'`.

### `Could not dynamically require "...better_sqlite3.node"`

Vite a tenté de bundlifier le module natif `better-sqlite3`. Ouvrir `vite.main.config.ts` et s'assurer qu'il contient :

```
import { defineConfig } from 'vite';

export default defineConfig({
  build: {
    rollupOptions: {
      external: [
        'better-sqlite3',
        /\.node$/,
      ],
    },
  },
});
```

### Les boutons ne répondent pas (aucune erreur visible)

Le preload est probablement vide ou mal configuré — `window.api` n'est pas exposé. Vérifier que `src/preload/preload.ts` contient bien l'appel `contextBridge.exposeInMainWorld('api', { ... })` (voir Étape 10).

## Résumé des concepts clés

| Concept | Rôle | Version |
|---|---|---|
| **Main Process** | Node.js — accès fichiers/DB | Toutes |
| **Renderer** | Angular — interface utilisateur | Toutes |
| **IPC** | Communication entre les deux processus | Toutes |
| **Preload** | Pont sécurisé via `contextBridge` | Toutes |
| **Signal** | Valeur réactive Angular | Toutes |
| `@Input` | Donnée du parent vers l'enfant | Toutes |
| `@Output` | Événement de l'enfant vers le parent | Toutes |
| **SQLite** | Base de données dans un fichier | V2 + V3 |
| **ORM** | Couche d'abstraction entre code et SQL | V3 |
| **Prisma Schema** | Fichier décrivant les modèles — source de vérité | V3 |
| **Migration** | Fichier SQL versionné décrivant un changement de schéma | V3 |
| **Prisma Client** | API TypeScript typée pour interroger la base | V3 |
| **Electron Forge** | Outil de build et de packaging pour Electron | V3 |
| **Vite** | Bundler — compile ESM → CJS en un seul fichier | V3 |
| **Driver adapter** | Librairie JavaScript qui remplace le binaire natif Prisma | V3 |
| `prisma.config.ts` | Configuration de la CLI Prisma v7 (URL, chemin du schéma) | V3 |
| `provideZonelessChangeDetection()` | Détection de changements Angular sans zone.js | V3 |